

Redes Neuronales

CNN

Realizado por:
Nora Tian García Bonafuente

INTELIGENCIA ARTIFICIAL



ÍNDICE

Introducción.....	3
Descripción general del modelo	3
Arquitectura básica	4
Mecanismo de aprendizaje.....	6
Ventajas y desventajas de la CNN.....	8
Variantes del modelo	9
Casos de Uso	13
IMPLEMENTACIÓN PRÁCTICA DE LA CNN.....	14
Análisis del framework	14
Evaluación del rendimiento	15
Explicación del Código	15
Reflexión Final.....	20
Bibliografía	21

Introducción

Desde siglos, la sociedad ha ido evolucionando hasta nuestros días con el fin de mejorar la calidad de vida. Este deseo de progreso ha dado lugar a descubrimientos y avances que han cambiado por completo nuestra manera de vivir.

La combinación de la ciencia y la tecnología ha sido clave para llevar a cabo este progreso, lo cual, ha abierto nuevas puertas a posibilidades, que antes, ni se habían planteado o imaginado.

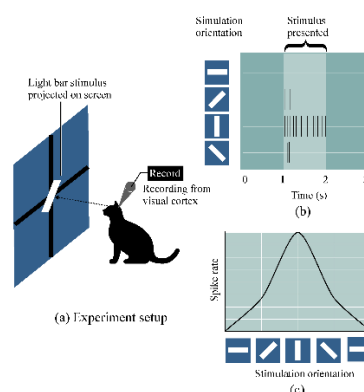
En la actualidad, la Inteligencia Artificial surge como una de las disciplinas más revolucionarias, transformando la manera en que las personas interactúan con el mundo que les rodea.

Dentro de las distintas áreas de la IA, las redes neuronales tienen un papel especial. Inspiradas en cómo funciona nuestro cerebro, estas redes buscan imitar los procesos de aprendizaje y pensamiento humano. Gracias a su capacidad para analizar datos, identificar patrones y tomar decisiones por su cuenta, se han convertido en una pieza fundamental para resolver problemas complejos en muchos campos, entre ellos, la medicina, economía, tecnología y las ciencias sociales.

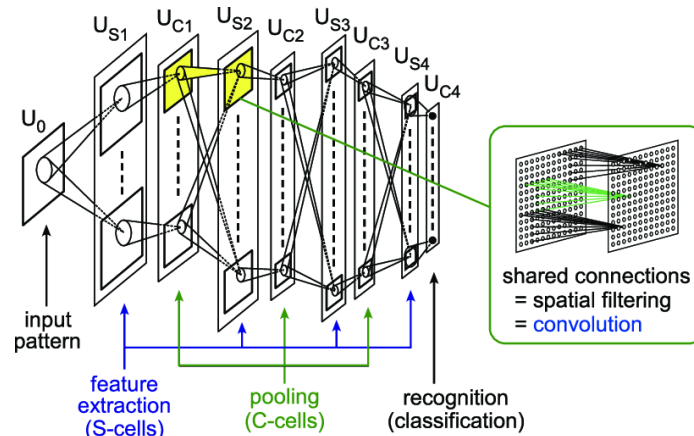
Descripción general del modelo

Las redes neuronales convolucionales forman parte del [machine learning](#), y son el centro de los algoritmos del [deep learning](#). Las CNN procesan datos para la clasificación de imágenes y el reconocimiento de objetos.

Retrocediendo a los años 60, dos neurocientíficos, Hubel y Wiesel, realizaron un [experimento](#) con gatos, que revolucionó la comprensión de la visión biológica. Descubrieron que ciertas neuronas de la corteza visual del cerebro se activaban, sólo, ante estímulos visuales concretos. Este hallazgo sentó bases para crear modelos artificiales que imitaran este comportamiento, detectando patrones visuales básicos.



El japonés **Kunihiko Fukushima** inspirado en estos descubrimientos, en 1980 desarrolló un modelo llamado **Neocognitron**, considerado el primer antecedente de las redes convolucionales modernas. Fue en este momento donde se estableció la arquitectura jerárquica de capas.



Más adelante, en 1989, el científico **Yann LeCun** logró entrenar una CNN completa, utilizando retro propagación. Su modelo llamado **LeNet-5** fue diseñado para reconocer dígitos escritos a mano. LeNet-5 combinaba las capas convolucionales, de pooling y densas, marcando un hito en el aprendizaje automático supervisado.

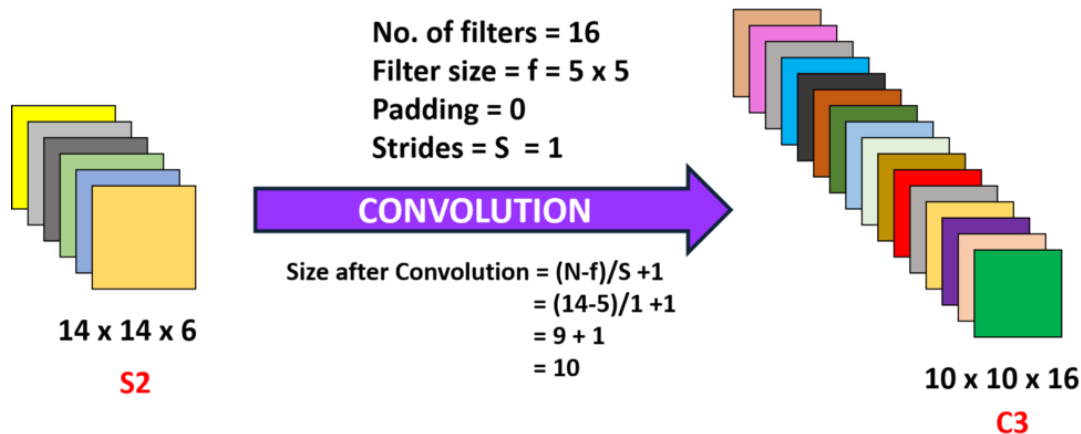
Pero no fue hasta 2012 con la aparición de **AlexNet**, desarrollada por **Alex Krizhevsky** en colaboración con **Geoffrey Hinton** e **Ilya Sutskever**, lograron una mejora sin precedentes en la tarea de clasificación visual.

Arquitectura básica

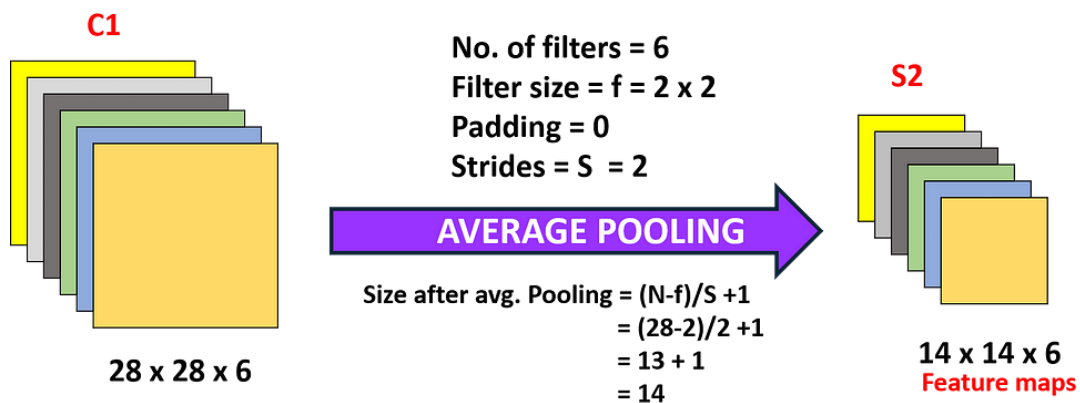
Su estructura interna está compuesta por 3 capas fundamentales:

- Capa de entrada.
- Capas convolucionales y pooling.
- Capa de clasificación.
- Capa de salida.

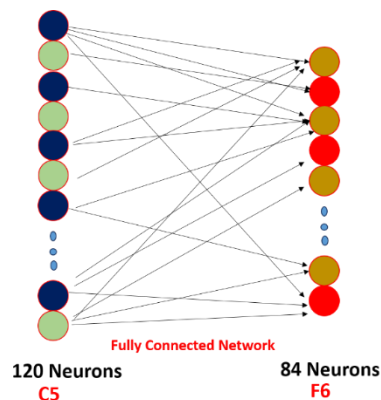
Estas capas trabajan en conjunto para analizar y procesar la información que hay en una imagen. El proceso comienza con la **capa de entrada**, que recibe los datos en forma de matriz de píxeles. Puede incluir múltiples canales, como RGB en imágenes a color. A continuación, le siguen las **capas convolucionales**, que se encargan de detectar automáticamente patrones visuales. En estas capas se aplican pequeños filtros que se deslizan por la imagen original, recogiendo características como bordes, formas o texturas. Como resultado, se crean los llamados mapas de activación, que representan las zonas donde esos patrones aparecen con mayor intensidad.



Luego, con la **capa pooling** lo que se hace es reducir el tamaño de los mapas de activación, conservando la información más relevante. Esto se consigue mediante operaciones como el max pooling, que coge el valor más alto dentro de pequeñas regiones del mapa, así, la red se vuelve más eficiente disminuyendo el riesgo de sobreajuste y convirtiéndose más resistente a pequeñas variaciones o desplazamientos en la imagen.

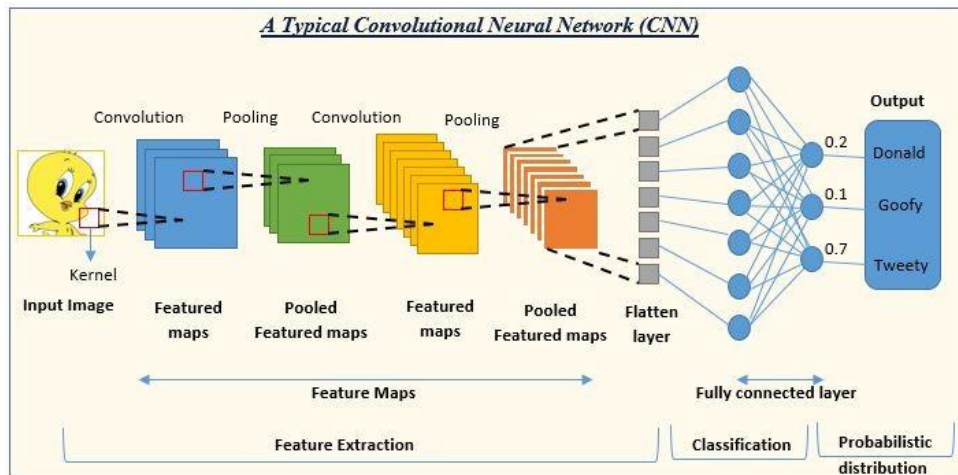


Después de varias combinaciones de capas convolucionales y de pooling, se pasa a la **capa de clasificación** y salida, donde toda la información que ha ido recogiendo durante el proceso, se transforma en un vector unidimensional. Este vector se conecta a una o varias neuronas encargadas de realizar la predicción.



La **capa de salida** es la última etapa de la red, y su función es generar la predicción final, transformando la información procesada por las capas anteriores en un resultado interpretable. Funciones como *softmax* calculan la probabilidad de que la imagen pertenezca a una clase u otra, completando así el proceso de análisis y clasificación.

Aquí se muestra un ejemplo del proceso entero de la CNN con una imagen.



Mecanismo de aprendizaje

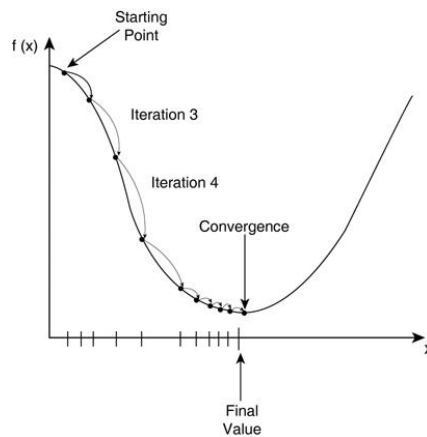
El aprendizaje de una CNN se basa en el proceso de entrenamiento mediante el cual, la red, ajusta sus parámetros internos para mejorar su rendimiento.

Todo este proceso se basa en una serie de pasos identificados, como:

- Proceso de entrenamiento
- Función de pérdida
- Retro propagación del error o Backpropagation
- Optimización

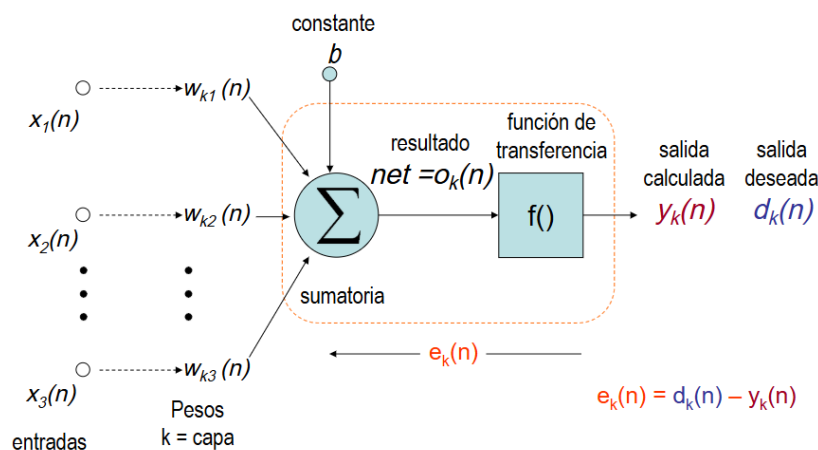
El proceso comienza con la **fase de entrenamiento** donde, a la red, se le muestra una gran cantidad de datos etiquetados. La red analiza cada imagen y realiza una predicción inicial. Y luego compara su resultado con la respuesta esperada.

Para saber qué tan buena o mala ha sido esa predicción, se utiliza la **función de pérdida**, que mide el error cometido. Esta función devuelve un valor numérico y cuanto mayor es el valor, mas ha fallado; y cuanto mas se acerque a cero, mejor ha sido la predicción.



Una vez calculado el error, se pasa al algoritmo de **retro propagación**. Este mecanismo se encarga de repartir ese error hacia atrás, a través de todas las capas de la red, calculando así, la contribución de cada peso o conexión al resultado final. Con esto, lo que se logra es ajustar los pesos de la red para aprender de sus errores, conociendo en que partes del modelo deben de corregirse para mejorar el rendimiento.

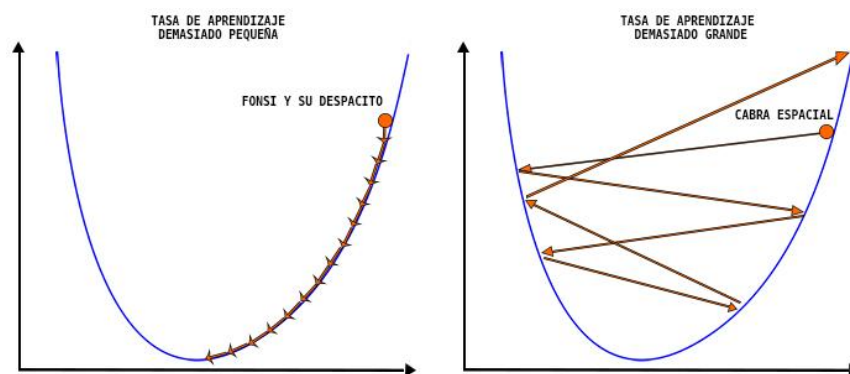
Retropropagación



Y este ajuste de pesos se hace a través de la **optimización**, que usa algoritmos como el descenso del gradiente. Este proceso modifica poco a poco los pesos, con el fin de minimizar el valor de la función de pérdida. Es decir, intenta que la red cometa menos errores.

Dentro de este proceso también se incluye un parámetro llamado **tasa de aprendizaje** (learning rate), que determina la magnitud de estos ajustes. Un learning rate grande provoca cambios más rápidos en los pesos, mientras que uno pequeño hace que los ajustes sean más suaves y lentos.

Elegir una tasa de aprendizaje grande o pequeña no es necesariamente bueno o malo: dependerá del objetivo. Si se quiere entrenar rápidamente en pocas iteraciones, puede ser útil un learning rate mayor (aunque con más riesgo de inestabilidad). Si se cuenta con más tiempo o se busca más precisión, uno menor puede funcionar mejor. Eso sí, hay que tener en cuenta que el tiempo de entrenamiento puede variar mucho dependiendo del valor elegido.



A lo largo de muchas iteraciones o épocas, la red mejora su capacidad para aprender patrones útiles para la predicción correcta de datos.

Ventajas y desventajas de la CNN

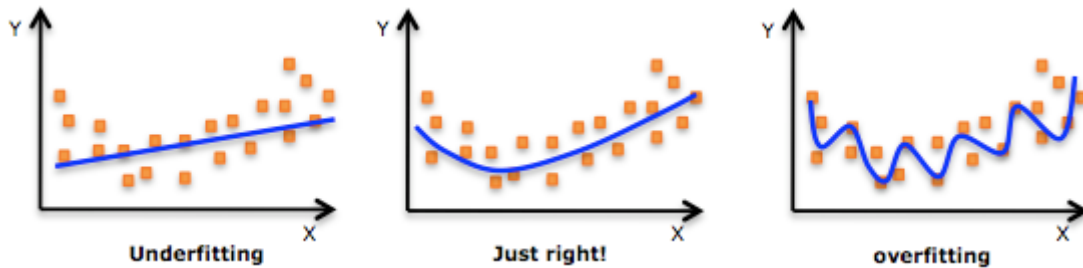
Entre sus principales beneficios, es la capacidad de reconocer patrones espaciales dentro de los datos, lo que las hace eficientes en el procesamiento de imágenes y vídeos. Las CNN, además, aprovechan estructuras como los filtros y la compartición de pesos para reducir gradualmente el número de parámetros, lo que permite una mayor eficiencia computacional.

Asimismo, estas redes son capaces de aprender automáticamente las características relevantes de los datos sin necesidad de intervenir manualmente, lo que genera simplicidad en una amplia variedad de tareas. También destacan por su relativa invariancia a pequeñas traslaciones de los objetos dentro de las imágenes, lo que mejora su capacidad de generalización.

Sin embargo, las redes neuronales convolucionales también tienen sus desventajas. Entre ellas estaría a su alta dependencia de uso de grandes cantidades de datos etiquetados para lograr un buen rendimiento, lo que puede resultar costosa en términos computacionales.

Finalmente, si no se usan técnicas para evitar el sobre ajuste, la red podría memorizar los datos de entrenamiento en vez de generalizar a datos nuevos.

El sobre ajuste ocurre cuando la red neuronal aprende excesivamente los detalles de los datos durante el entrenamiento, incluyendo ruido o cosas específicas, en lugar de captar patrones generales que puedan aplicarse a datos nuevos. Esto muestra que el modelo tiene un rendimiento excelente en el conjunto de entrenamiento, pero falla al predecir sobre datos de prueba o en situaciones reales.

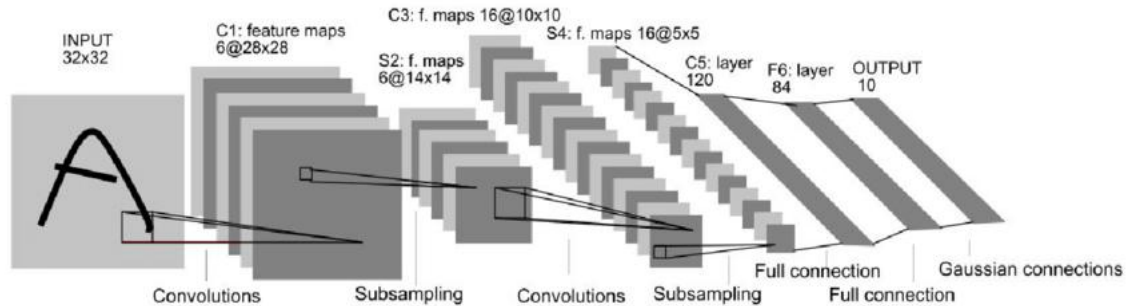


Para ello, existen varios métodos para evitar el sobreajuste. Entre ellos, la generación de datos sintéticos mediante **aumentación**, que amplía el conjunto de datos de entrenamiento y mejora la diversidad. También está **dropout** para reducir la complejidad del modelo; la **normalización** de los datos de entrada, asegurando escalas consistentes para una mejor interpretación de patrones; o **validación cruzada**, dividiendo los datos en conjuntos de entrenamiento y prueba para garantizar resultados fiables. Y por último se encuentra el método de **early stopping**, que detiene el entrenamiento cuando el error del conjunto de validación comienza a aumentar, ayudando a preservar la capacidad de generalización.

Variantes del modelo

A lo largo del tiempo, han surgido diferentes arquitecturas que modifican o amplían la idea original de las CNN, dando lugar a distintos tipos de redes neuronales convolucionales, buscando mejorar el rendimiento, eficiencia y capacidad de generalización del modelo.

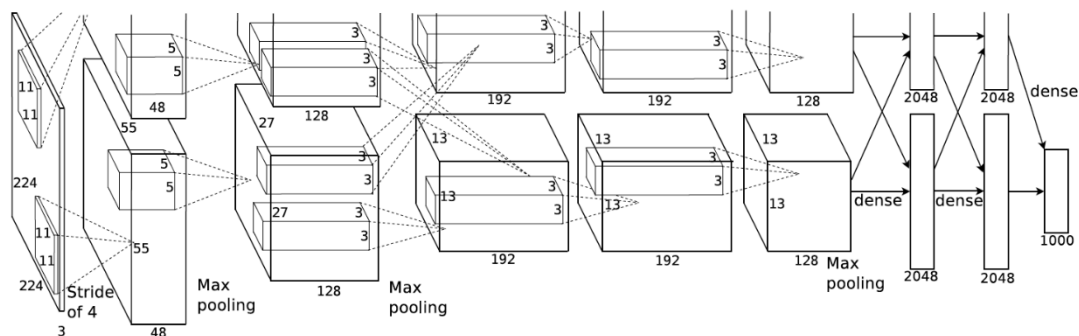
Una de las primeras arquitecturas relevante fue **LeNet-5**, desarrollada en 1988, que sentó las bases de las CN modernas, y fue usada principalmente para el reconocimiento de dígitos manuscritos.



Posteriormente, en 2012, apareció el modelo **AlexNet**. Su arquitectura se compone de ocho capas, de las cuales cinco son convolucionales y tres completamente conectadas (fully connected).

Una de las características más innovadoras de este modelo, fue el uso de la función de activación ReLU (Rectified Linear Unit), que permitió un entrenamiento más rápido en comparación con las funciones de activación sigmoide o la tangente hiperbólica. Además, introdujo el uso de dropout, una técnica de regularización que apaga aleatoriamente ciertas neuronas durante el entrenamiento para evitar el sobreajuste. Y otra de sus aportaciones fue el uso de GPY para entrenar redes profundas permitiendo reducir el tiempo de entrenamiento.

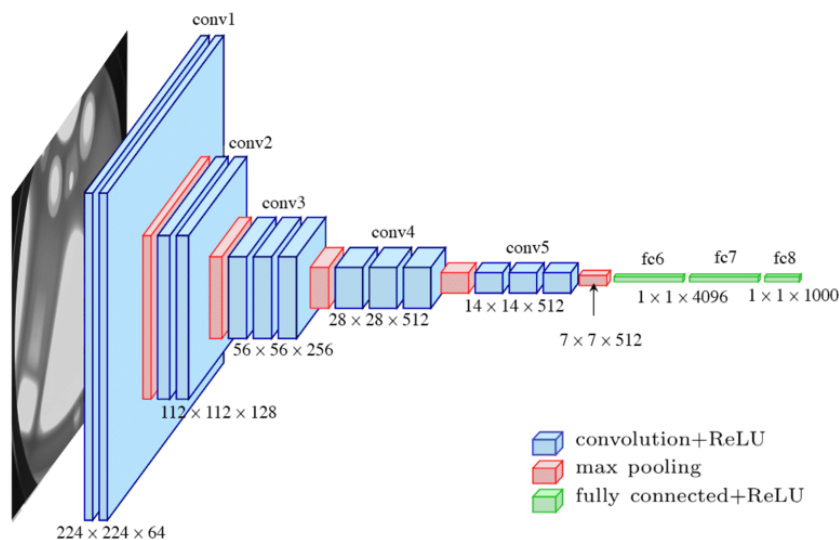
Asimismo, AlexNet incorporó técnicas como normalización local de respuesta y el uso de superposición (overlapping) en el max pooling, mejorando así, la capacidad de generalización de la red.



Por su parte, **VGGNet**, desarrollada por el Visual Geometry Group de la Universidad de Oxford en 2014 apostaba por una arquitectura profunda pero simple y uniforme, usando sólo convoluciones con filtros pequeños de tamaño 3x3, junto con capas de pooling 22x2 a lo largo de toda la red.

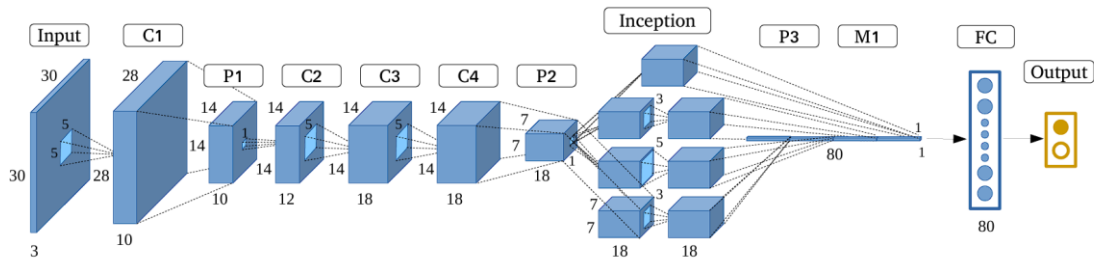
Una de las principales aportaciones de este modelo fue demostrar que al incremental la profundidad de la red, podría mejorar el rendimiento en tareas de clasificación, siempre que se mantuviese una estructura regular y bien diseñada.

No obstante, este incremento en la profundidad trajo consigo un mayor número de parámetros, concretamente en las capas fully connected del final de la red. A pesar de su efectividad, requiere mayor capacidad computacional y recursos de memoria, lo que puede ser una limitación en ciertos contextos.

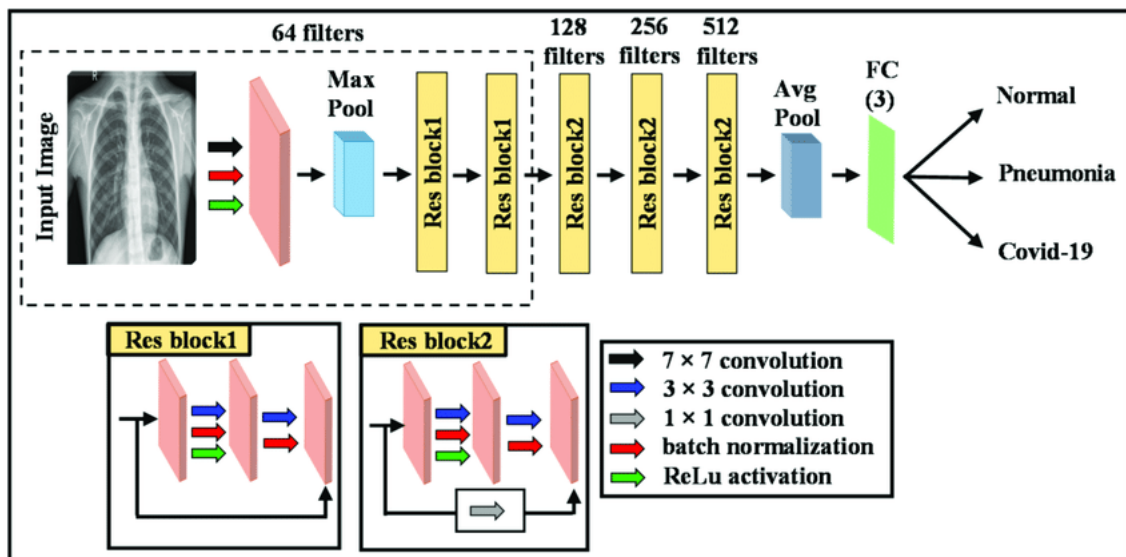


Posteriormente, **GoogLeNet** también conocida como Inception y **ResNet**, marcaron una nueva etapa en la evolución de las redes CNN al abordar dos limitaciones clave de sus predecesoras: el elevado número de parámetros y las dificultades relacionadas al entrenamiento de redes profundas.

La primera, desarrollada por Google en 2014, introdujo el concepto de módulos Inception, que combinan en paralelo convoluciones de diferentes tamaños junto con operaciones de pooling. Lo que se consiguió con esto fue la extracción de información a múltiples escalas dentro de cada capa, incrementando la riqueza de las características aprendidas sin aumentar significativamente la complejidad computacional.



Por su parte, ResNet (Redes Residuales), creada por Microsoft en 2015, solucionó el problema de la degradación del rendimiento al aumentar la profundidad de la red. Para ello, incorporó las llamadas conexiones residuales que permiten que la información fluya directamente entre capas saltando un o más etapas. Este diseño no sólo mejoró la capacidad de generalización, sino que también facilitó el proceso de entrenamiento y permitió el desarrollo de redes extremadamente profundas.



En resumen, las variantes arquitectónicas de las CNN han ido evolucionando con el objetivo de mejorar la eficiencia, profundidad y la capacidad de aprendizaje de estos modelos. No obstante, existen otras arquitecturas como **DesNet**, **MobileNet**, **EfficientNet** entre otras, pero las aquí presentadas son las más destacadas, que han servido como base para gran parte de las investigaciones y desarrollos actuales en visión por computados.

Casos de Uso

Las CNN han demostrado ser eficaces en una amplia variedad de aplicaciones prácticas, especialmente en aquellas relacionadas con la visión artificial. Un ejemplo destacado es el del reconocimiento de imágenes, donde estas redes permiten clasificar objetos, personas o escenas con gran precisión. Esta tecnología se utiliza en plataformas de redes sociales para el etiquetado automático de fotos, como ocurre en Facebook o Google Photos.

Otro ámbito importante es el de la **conducción autónoma**, donde las CNN procesan imágenes captadas por cámaras instaladas en los vehículos para identificar señales de tráfico, peatones, otros coches o carriles, permitiendo decisiones en tiempo real. Un ejemplo de su uso se encuentra en ciudades como San Francisco, donde ya circulan coches sin conductor ([EuroNews](#)).

En la **medicina**, estas redes se han aplicado a la detección automática de tumores en radiografías, mamografías y resonancias magnéticas, lo que agiliza y mejora el diagnóstico clínico. Según un estudio, las mamografías asistidas por inteligencia artificial aumentan la detección del cáncer de mama en un 20% ([BreastCancer.org](#)).

Las CNN también se utilizan en la **agricultura**, donde son aplicadas para analizar imágenes de cultivos y detectar enfermedades, plagas o deficiencias nutricionales. Esta práctica permite avanzar hacia una agricultura más sostenible y eficaz ([Cadena SER](#)).

En el campo de la **seguridad y vigilancia**, las CNN permiten el reconocimiento facial y el análisis de comportamiento. En Madrid, por ejemplo, se ha ampliado la red de cámaras con inteligencia artificial a la Plaza del Dos de Mayo, permitiendo rastrear personas u objetos según sus características y detectar infracciones de forma automatizada ([El País](#)).

En la **industria manufacturera**, estas redes se utilizan en el control de calidad automatizado, inspeccionando productos durante la producción para detectar defectos de forma precisa y rápida ([Redalyc](#)).

A estas aplicaciones se suman desarrollos más recientes y dirigidos al **hogar inteligente**, como las bombillas con reconocimiento facial ([Pr Newswire](#)) o cámaras como la Nest Cam de Google. ([ElEspañol](#))

Estos dispositivos emplean CNN para detectar rostros, movimientos u objetos, y activar funciones personalizadas según la presencia o identidad de las personas. Gracias a estas capacidades, es posible mejorar la seguridad del hogar y automatizar tareas de manera más intuitiva y eficiente.

En conjunto, estos ejemplos reflejan cómo las redes neuronales convolucionales han trascendido el ámbito académico para convertirse en herramientas esenciales en sectores tan diversos como la salud, la automoción, la agricultura, la seguridad, la industria y los hogares conectados.

IMPLEMENTACIÓN PRÁCTICA DE LA CNN

Análisis del framework

En este proyecto, he implementado una red neuronal convolucional desde cero, para clasificar imágenes del conjunto de datos [CIFAR-10](#). Este dataset contiene imágenes con 10 categorías de objetos, como gatos, perros, aviones, coches etc. El objetivo principal es entrenar un modelo que pueda reconocer estas imágenes y clasificarlas.

Antes de comenzar con la implementación de la CNN, lo primero que hice fue instalar las librerías necesarias para el proyecto. Esto incluía instalar los paquetes esenciales para las redes neuronales como TensorFlow o Pytorch, además de las librerías auxiliares como Numpy, Matplotlib, etc.

Fue en este paso donde me encontré problemas con la instalación de las librerías Tensorflow y Pythorch. En concreto, al intentar instalar Tensorflow, tuve que recurrir al comando “pip install tensorflow-cpu” dado que tuve problemas relacionados con la compatibilidad del hardware con la versión para GPU. No obstante, después de instalar esta versión, me daba errores.

Como solución, decidí al final utilizar Google Colab como entorno de trabajo. Esta plataforma, basada en la nube, ya tiene integrado las librerías necesarias; por lo que pude centrarme en el desarrollo e implementación del modelo.

Elegí Tensorflow por su flexibilidad para diseñar la CNN desde cero. Además, su compatibilidad con Google Colab, simplificaron bastante el desarrollo. En cuanto a Matplotlib, es una librería estándar para cualquier proyecto, debido a la facilidad de uso.

Las ventajas de utilizar estas librerías y el entorno de desarrollo son que ofrecen herramientas avanzadas como Early Stopping y optimizadores, lo que me permitió mejorar el rendimiento del modelo y evitar el sobreajuste durante el entrenamiento.

Con el entorno ya preparado, quise comenzar con una estructura básica, y progresivamente, ir añadiendo complejidad al modelo. Por tanto, como primera prueba, utilicé una arquitectura básica de tres capas convolucionales, seguidas de las capas de max pooling para reducir la dimensión de las imágenes.

El objetivo es conseguir un modelo funcional y que aprenda las características de las imágenes.

Evaluación del rendimiento

Para evaluar el rendimiento del modelo, se utilizaron dos métricas principales. La precisión (accuracy), que mide el porcentaje de imágenes bien clasificadas. Por otro lado, la pérdida (loss) indica la distancia entre las predicciones realizadas y los valores correctos durante el proceso de entrenamiento y validación.

El modelo alcanzó una precisión de la prueba de aproximadamente 78,69% con una pérdida del 62,69%. A lo largo de las iteraciones o épocas, pude ver que hubo una mejora progresiva tanto en la precisión como en la reducción de la pérdida, hasta que la función de EarlyStopping detuvo el entrenamiento al no observar mejoras en la validación.

En las gráficas, se puede ver una clara convergencia del modelo, sin datos evidentes de sobreajuste gracias al dropout y la normalización.

Explicación del Código

Ahora voy a explicar las fases claves que he ido siguiendo para el desarrollo de la CNN.

Hay que comenzar importando las librerías necesarias, así como de cargar y preparar los datos. El dataset utilizado ha sido CIFAR-10, donde se ha dividido automáticamente en datos de entrenamiento y datos de prueba.

Es importante que los datos estén bien preparados, para que el modelo pueda aprender patrones útiles. Por esto, normalizar los valores de los píxeles de un rango de [0,255] a [0,1], ayuda a la red neuronal a aprender de forma más estable y rápida.

```
# 2. Cargar y preparar los datos
print("Cargando dataset CIFAR-10...")
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.cifar10.load_data()
print(f"Datos cargados: {train_images.shape[0]} imágenes de entrenamiento, {test_images.shape[0]} de prueba.")

# Normalización
train_images = train_images.astype('float32') / 255.0
test_images = test_images.astype('float32') / 255.0

# Mostrar algunas imágenes con sus etiquetas
class_names = ['Avión', 'Auto', 'Pájaro', 'Gato', 'Ciervo', 'Perro', 'Rana', 'Caballo', 'Barco', 'Camión']
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(train_images[i])
    plt.title(class_names[int(train_labels[i])])
    plt.axis('off')
plt.suptitle("Primeras imágenes del set de entrenamiento")
plt.tight_layout()
plt.show()
```

Cargando dataset CIFAR-10...
 Datos cargados: 50000 imágenes de entrenamiento, 10000 de prueba.
 <ipython-input-10-edd92e149647>:16: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error
 plt.title(class_names[int(train_labels[i])])

Primeras imágenes del set de entrenamiento



Para evitar problemas de sobreajuste, utilicé Data Augmentation para generar nuevas imágenes artificiales aplicando transformaciones. Éstas incluyen rotación, desplazamientos y giros.

```
# 3. Data Augmentation
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
datagen.fit(train_images)

# Mostrar ejemplos de augmentación
print("Ejemplos de Data Augmentation:")
sample = train_images[1].reshape((1,) + train_images[1].shape)
plt.figure(figsize=(10, 2))
for i, batch in enumerate(datagen.flow(sample, batch_size=1)):
    plt.subplot(1, 5, i + 1)
    plt.imshow(batch[0])
    plt.axis('off')
    if i == 4:
        break
plt.suptitle("Variaciones de una imagen con Data Augmentation")
plt.show()
```

Variaciones de una imagen con Data Augmentation



Lo más importante viene ahora, que es la definición del modelo de la red. Como ya sabemos, la arquitectura se basa en diferentes capas. Por un lado, estarían las capas convolucionales, en este caso se han utilizado tres, seguidas de Batch Normalization y MaxPooling. Con Batch Normalization, lo que se consigue es estabilizar el proceso de aprendizaje, al normalizar la activación de las capas, y luego se reduce la dimensión espacial de los datos con MaxPooling.

Generalizando la estructura de capas, ahora vendrían las densas, con el fin de procesar los datos extraídos y hacer las predicciones.

Estas incluyen capas como **Flatten**, que transforma los datos 3d a un vector unidimensional. Luego capas con activación **ReLU** que actúa como capa oculta responsable de las predicciones, mientras que **Dropout** desactiva aleatoriamente el 50% de las neuronas para reducir el sobreajuste. Y ya para terminar, estaría la capa Dense con activación **Softmax** funciona como la salida del modelo, con 10 neuronas que representan las clases del conjunto de datos y calculan la probabilidad de pertenencia a cada categoría.

```
# 4. Definición del modelo CNN
print("Definiendo el modelo CNN...")
model = models.Sequential([
    layers.Conv2D(64, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(256, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])
model.summary()
```

Definiendo el modelo CNN...

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:10: in __init__
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 30, 30, 64)	1,792
batch_normalization_3 (BatchNormalization)	(None, 30, 30, 64)	256
max_pooling2d_3 (MaxPooling2D)	(None, 15, 15, 64)	0
conv2d_4 (Conv2D)	(None, 13, 13, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 13, 13, 128)	512
max_pooling2d_4 (MaxPooling2D)	(None, 6, 6, 128)	0
conv2d_5 (Conv2D)	(None, 4, 4, 256)	295,168
batch_normalization_5 (BatchNormalization)	(None, 4, 4, 256)	1,024
max_pooling2d_5 (MaxPooling2D)	(None, 2, 2, 256)	0
flatten_1 (Flatten)	(None, 1024)	0
dense_2 (Dense)	(None, 512)	524,800
dropout_1 (Dropout)	(None, 512)	0
dense_3 (Dense)	(None, 10)	5,130

Total params: 902,538 (3.44 MB)
Trainable params: 901,642 (3.44 MB)
Non-trainable params: 896 (3.50 KB)

Una vez establecida la estructura de la neurona, se compila el modelo. Para ello, he usado Adam, un optimizador con un learning rate de 0.0005 para una convergencia más fina y estable.

La función de pérdida que he utilizado es la entropía cruzada.

Otra manera de evitar que se sobreentrene el modelo es utilizando la función de EarlyStopping, que detiene el proceso de entrenamiento si la pérdida en validación no mejora tras 3 épocas.

```
# 6. EarlyStopping
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)
print("EarlyStopping configurado (patience=3).")
```

Ahora hay que entrenar el modelo, para ello, establecí 20 épocas, aunque podría finalizar el proceso antes, si se activa early stopping.

```
# 7. Entrenamiento
print("Entrenando modelo...")

history = model.fit(
    datagen.flow(train_images, train_labels, batch_size=64),
    epochs=20,
    validation_data=(test_images, test_labels),
    callbacks=[early_stopping],
)

# Mostrar evolución en tabla
df = pd.DataFrame(history.history)
df['epoch'] = df.index + 1
print("Resumen del entrenamiento:")
display(df[['epoch', 'accuracy', 'val_accuracy', 'loss', 'val_loss']].round(4).tail(5))
```

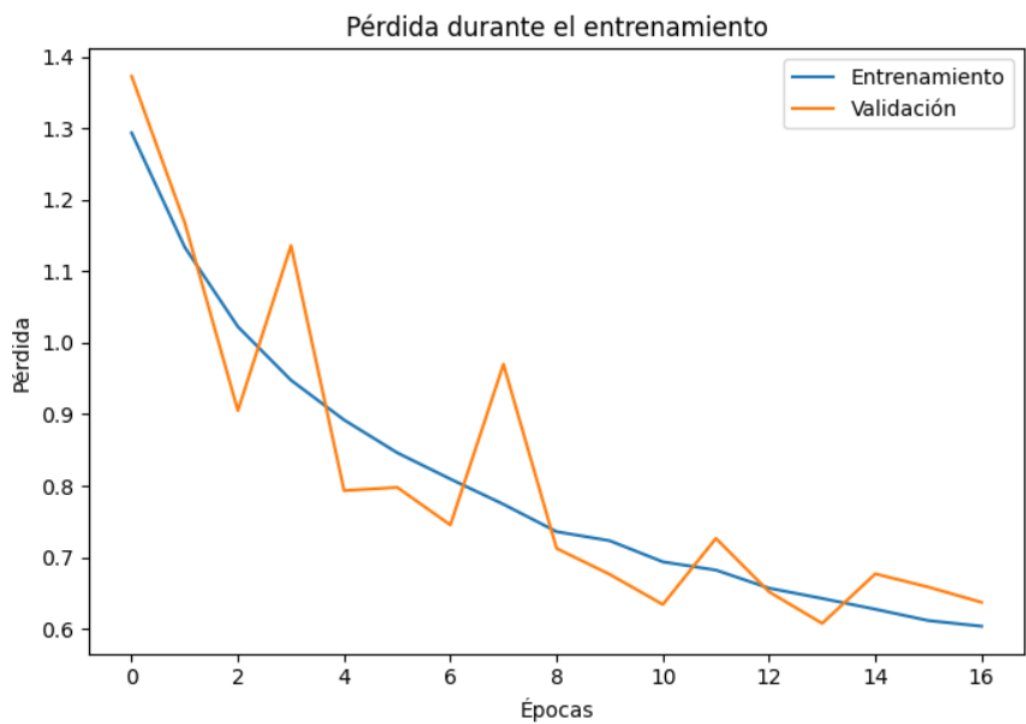
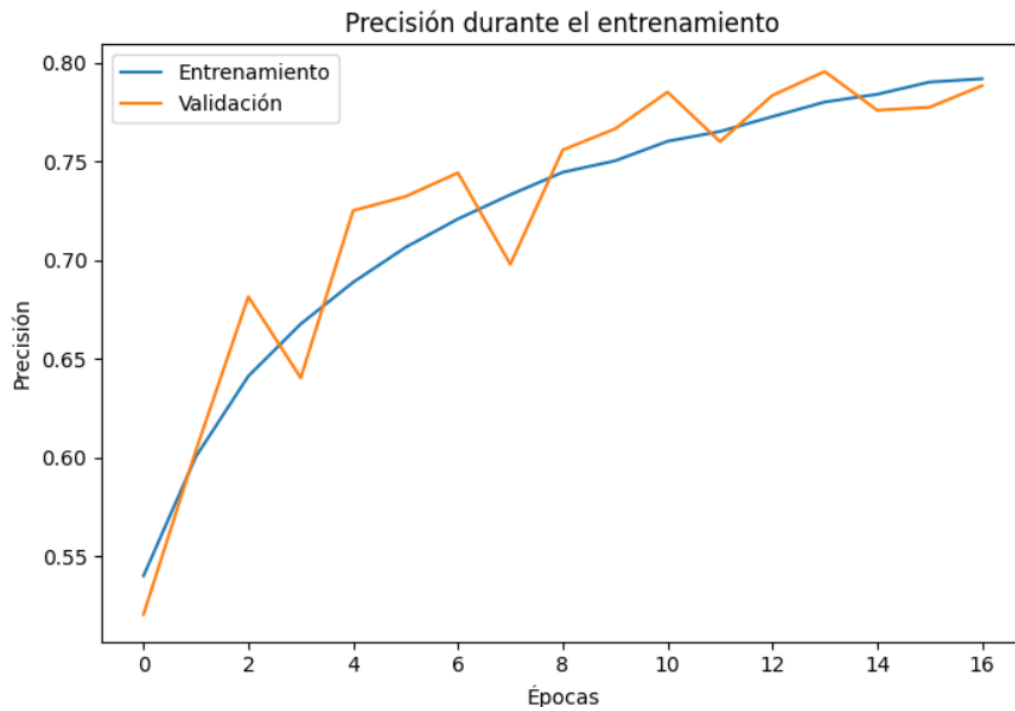
Resumen del entrenamiento:

	epoch	accuracy	val_accuracy	loss	val_loss
12	13	0.7728	0.7835	0.6568	0.6517
13	14	0.7802	0.7956	0.6424	0.6076
14	15	0.7841	0.7760	0.6274	0.6768
15	16	0.7903	0.7775	0.6115	0.6584
16	17	0.7919	0.7884	0.6036	0.6370

Tras completar el entrenamiento, hay que evaluar el rendimiento final del modelo en el conjunto de prueba, así se puede saber si ha aprendido de forma eficaz. Para ello, generé unos gráficos, donde se muestra tanto la precisión como la pérdida en los conjuntos de entrenamiento y validación.



✓ Evaluación final:
Precisión: 0.7956
Pérdida: 0.6076



Reflexión Final

Hacer este proyecto me ha servido para entender cómo funcionan las redes neuronales artificiales, en concreto, la CNN. He de admitir, que cuando empecé, no sabía cómo hacerlo; hasta que fui buscando documentación para la parte de teoría del principio.

Aunque la práctica es fundamental para demostrar el funcionamiento de una red neuronal, es muy importante tener conocimientos previos, para familiarizarte con el tema.

A lo largo de este trabajo, he aprendido a diseñar e implementar una arquitectura de red. Tengo que decir que me he encontrado con varios problemas, sobre todo con la instalación de paquetes, pero, he sabido encontrar alternativas para poder desarrollar la red.

Me he dado cuenta de que es importante adaptarse a las situaciones que se dan, y buscar todas las soluciones posibles.

También, he aprendido a evaluar el rendimiento del modelo más allá de la precisión final, analizando las gráficas y observando el comportamiento del modelo a lo largo de las distintas épocas. Implementar mecanismos para evitar el sobreajuste me ha ayudado a ver cómo influye de manera directa sobre la estabilidad del entrenamiento.

Más allá de los aspectos técnicos, he tomado conciencia del enorme avance que representan las redes neuronales artificiales para la ciencia y la tecnología.

Aplicaciones como el reconocimiento de imágenes, la medicina asistida por IA o los sistemas autónomos se apoyan en estos principios. Es motivador saber que los conocimientos adquiridos en este proyecto son los mismos o similares a los que se aplican en desarrollos punteros en la industria y la investigación.

En fin, me ha encantado llevar a cabo este proyecto, y ahora tengo aún más claro que quiero dedicarme a esta área de la Inteligencia Artificial, sea cual sea su enfoque. Todas me fascinan.

Bibliografía

Lov Technology. (s.f.). *Redes neuronales convolucionales (CNN)*. Lov Technology.
<https://lovtechnology.com/redes-neuronales-convolucionales-cnn/>

Ichi.pro. (s.f.). *La breve historia de las redes neuronales convolucionales*.
<https://ichi.pro/es/la-breve-historia-de-las-redes-neuronales-convolucionales-76620623015807>

Codificando Bits. (s.f.). *Redes convolucionales: GoogLeNet y ResNet*.
<https://codificandobits.com/curso/fundamentos-deep-learning-python/redes-convolucionales-15-googlenet-resnet/>

Telefónica. (s.f.). *Redes neuronales convolucionales: qué son, tipos y aplicaciones*.
<https://www.telefonica.com/es/sala-comunicacion/blog/redes-neuronales-convolucionales-que-son-tipos-aplicaciones/>

iArtificial. (s.f.). *Cómo implementar una red neuronal convolucional en Python*.
<https://iartificial.blog/desarrollo/como-implementar-una-red-neuronal-convolucional-en-python/>

Aprende Machine Learning. (s.f.). *Clasificación de imágenes en Python*.
<https://www.aprendemachinelearning.com/clasificacion-de-imagenes-en-python/>

INTELIGENCIA ARTIFICIAL

5 MAYO DE 2025